

构造、非传统、随机与近似算法

wsy

2026 年 8 月 3 日

目录

构造选讲

非传统题选讲

交互选讲

通信选讲

提交答案选讲

随机与近似算法选讲

CF1667C Half Queen Cover

有一个 $n \times n$ 的棋盘。棋盘上一个位置的皇后可以攻击与它同行、同列和同一条主对角线上的格子。

问最少放多少个皇后，可以攻击到整个棋盘。输出一种方案。

$1 \leq n \leq 10^5$ 。

CF1667C Half Queen Cover

先来分析答案下界。如果放了 k 个皇后，那么通过行列最多能攻击 k 行 k 列。剩下的一个 $(n - k) \times (n - k)$ 的部分只能用对角线来攻击。

CF1667C Half Queen Cover

先来分析答案下界。如果放了 k 个皇后，那么通过行列最多能攻击 k 行 k 列。剩下的一个 $(n - k) \times (n - k)$ 的部分只能用对角线来攻击。

这里一共有 $2(n - k) - 1$ 条对角线，因此必须要有
 $k \geq 2(n - k) - 1$ 解得 $3k \geq 2n - 1$ 。因此猜测答案为 $\lceil \frac{2n-1}{3} \rceil$ 。

CF1667C Half Queen Cover

先来分析答案下界。如果放了 k 个皇后，那么通过行列最多能攻击 k 行 k 列。剩下的一个 $(n - k) \times (n - k)$ 的部分只能用对角线来攻击。

这里一共有 $2(n - k) - 1$ 条对角线，因此必须要有
 $k \geq 2(n - k) - 1$ 解得 $3k \geq 2n - 1$ 。因此猜测答案为 $\lceil \frac{2n-1}{3} \rceil$ 。

对 $n \equiv 2 \pmod 3$ 的构造：把棋盘大概平均的分成三份，对前两份填副对角线即可。

CF1667C Half Queen Cover

先来分析答案下界。如果放了 k 个皇后，那么通过行列最多能攻击 k 行 k 列。剩下的一个 $(n - k) \times (n - k)$ 的部分只能用对角线来攻击。

这里一共有 $2(n - k) - 1$ 条对角线，因此必须要有
 $k \geq 2(n - k) - 1$ 解得 $3k \geq 2n - 1$ 。因此猜测答案为 $\lceil \frac{2n-1}{3} \rceil$ 。

对 $n \equiv 2 \pmod 3$ 的构造：把棋盘大概平均的分成三份，对前两份填副对角线即可。

剩下对情况在右下角补。

AGC066A Adjacent Difference

给定一个 $n \times n$ 的矩阵 a ，你可以花费 $|w|$ 的代价选择一个 (x, y) ，修改 $a_{x,y}$ 为 $a_{x,y} + w$ 。

你需要花费不超过 $\frac{1}{2}dn^2$ 的代价，使得这个矩阵满足对于任意两个相邻的位置，他们差的绝对值 $\geq d$ 。

$n \leq 500$, $1 \leq d \leq 1000$, $-1000 \leq a_{i,j} \leq 1000$ 。

AGC066A Adjacent Difference

考虑将所有的 $a_{x,y}$ 变成两种数： $\equiv 0 \pmod{2d}$ 与 $\equiv d \pmod{2d}$ ，如果将矩阵黑白染色后，黑格子全部满足其中一种，白格子全部满足其中一种，那么任意相邻两个各自的差一定 $\equiv d \pmod{2d}$ ，一定满足条件。

AGC066A Adjacent Difference

考虑将所有的 $a_{x,y}$ 变成两种数： $\equiv 0 \pmod{2d}$ 与 $\equiv d \pmod{2d}$ ，如果将矩阵黑白染色后，黑格子全部满足其中一种，白格子全部满足其中一种，那么任意相邻两个各自的差一定 $\equiv d \pmod{2d}$ ，一定满足条件。

考虑一个格子变成 $\equiv 0 \pmod{2d}$ 的最小代价与 $\equiv d \pmod{2d}$ 的最小代价之和一定是 d ，因此两种方式的代价之和 $= dn^2$ ，因此必有一种方式的代价之和 $\leq \frac{1}{2}dn^2$ ，直接枚举两种操作取较小的即可。复杂度 $O(n^2)$ 。

AGC025D Choosing Points

给定 n, d_1, d_2 , 在 $2n \times 2n$ 的网格图上选出 n^2 个整点, 使得两两的距离都不是 $\sqrt{d_1}$ 和 $\sqrt{d_2}$ 。

$1 \leq n \leq 300$, $d_1, d_2 \leq 2 \times 10^5$ 。

AGC025D Choosing Points

关键结论：给距离 $= \sqrt{d}$ 的点之间连边，构成二分图：

- ▶ $d \bmod 4 = 0$ ：此时点对 x, y 奇偶性分别相同，除 2 后归；
- ▶ $d \bmod 4 = 1$ ：此时点对 x, y 奇偶性恰好有一个不同，按照 $x + y$ 奇偶性分类；
- ▶ $d \bmod 4 = 2$ ：此时点对 x, y 奇偶性都不同，按 x 或 y 奇偶性分类；
- ▶ $d \bmod 4 = 3$ ：根本没有边。

AGC025D Choosing Points

关键结论：给距离 $= \sqrt{d}$ 的点对之间连边，构成二分图：

- ▶ $d \bmod 4 = 0$ ：此时点对 x, y 奇偶性分别相同，除 2 后归；
- ▶ $d \bmod 4 = 1$ ：此时点对 x, y 奇偶性恰好有一个不同，按照 $x + y$ 奇偶性分类；
- ▶ $d \bmod 4 = 2$ ：此时点对 x, y 奇偶性都不同，按 x 或 y 奇偶性分类；
- ▶ $d \bmod 4 = 3$ ：根本没有边。

因此直接根据 d_1, d_2 分别对 $4n^2$ 个点染色，找出颜色最多的一组点即可，个数必然 $\geq n^2$ 。

QOJ962 Thanks to MikeMirzayanov

给定一个长度为 n 的排列，一次操作可以将原排列分成若干段，然后将段 reverse，即选定端点位置 $d_1, d_2, \dots, d_k = n$ ，将排列变成

$$p_{d_{k-1}+1}, \dots, p_{d_k}, \dots, p_{d_1} + 1, \dots, p_{d_2}, p_1, \dots, p_{d_1}$$

你需要在 120 次操作之内将原序列排序。

$$1 \leq n \leq 20000.$$

QOJ962 Thanks to MikeMirzayanov

考虑分治，按照大小分为两半，小的往左边分治，大的往右边分治。问题变成了给定一个 01 序列（将小的数看作 0，大的数看作 1），你需要在合理的操作次数内将序列变为 $0, \dots, 0, 1, \dots, 1$ 。

QOJ962 Thanks to MikeMirzayanov

考虑分治，按照大小分为两半，小的往左边分治，大的往右边分治。问题变成了给定一个 01 序列（将小的数看作 0，大的数看作 1），你需要在合理的操作次数内将序列变为 $0, \dots, 0, 1, \dots, 1$ 。

考虑将这个 01 序列相同的数缩成一段，变成 $01010\dots010$ 的性质，然后通过如下方式划分： $0|10|1|01|0|10|1|01|\dots$ ，这样在 reverse 之后序列变成了 011100011100 ，不难发现可以继续将相同的缩起来，段数变为了原来的 $\frac{1}{3}$ ，因此可以在 $\log_3 len$ 的操作步数内将一个 01 序列进行排序。

QOJ962 Thanks to MikeMirzayanov

考虑分治，按照大小分为两半，小的往左边分治，大的往右边分治。问题变成了给定一个 01 序列（将小的数看作 0，大的数看作 1），你需要在合理的操作次数内将序列变为 $0, \dots, 0, 1, \dots, 1$ 。

考虑将这个 01 序列相同的数缩成一段，变成 $01010\dots010$ 的性质，然后通过如下方式划分： $0|10|1|01|0|10|1|01|\dots$ ，这样在 reverse 之后序列变成了 011100011100 ，不难发现可以继续将相同的缩起来，段数变为了原来的 $\frac{1}{3}$ ，因此可以在 $\log_3 len$ 的操作步数内将一个 01 序列进行排序。

对于分治的不同区间，这种操作方式是可以并行的。操作次数 $O(\log^2 n)$ 。

CF1054G New Road Network

你需要构造一棵 n 个点的树。

给定 m 个集合限制，表示该集合内的点在树上联通。

构造方案或输出无解。

$1 \leq n, m \leq 2000$ 。

CF1054G New Road Network

考虑给构造出来的树不断剥叶子，当前剥掉的叶子所在的集合必定是他的父亲的子集。而可以被忽略的集合是仅包含一个点的集合。

CF1054G New Road Network

考虑给构造出来的树不断剥叶子，当前剥掉的叶子所在的集合必定是他的父亲的子集。而可以被忽略的集合是仅包含一个点的集合。

因此大致思路就是每次找到一个 x, y ，满足 x 所在的集合是 y 所在集合的一个子集，然后令 x 接在 y 下面。随后删除 x 点，并更新所有仅剩下一个点的集合，这些集合现在已经没用了。

CF1054G New Road Network

考虑给构造出来的树不断剥叶子，当前剥掉的叶子所在的集合必定是他的父亲的子集。而可以被忽略的集合是仅包含一个点的集合。

因此大致思路就是每次找到一个 x, y ，满足 x 所在的集合是 y 所在集合的一个子集，然后令 x 接在 y 下面。随后删除 x 点，并更新所有仅剩下一个点的集合，这些集合现在已经没用了。

考虑如何模拟上述过程，首先判断一个集合是否是另一个集合的子集可以使用 bitset。找到 x, y 并删除 x 后，所有可能失效的集合中一定只存在了 y 一个元素，因此在删除 x 后只需要重新算 y 与其他点的关系即可。复杂度 $O(\frac{n^2 m}{\omega})$ 。

AGC029F Construction of a tree

给定 $n - 1$ 个集合，你需要从每个集合中选择两个元素，在这两个元素之间连一条无向边。

要求最后 $n - 1$ 条无向边构成一棵树，或报告无解。

$$2 \leq n \leq 10^5, \sum |s_i| \leq 2 \times 10^5.$$

AGC029F Construction of a tree

首先考虑如果存在 x 个集合，他们的并的大小是 $\leq x$ ，那么必然无解。考虑这 x 个集合的并的点的子集在树上的连边数量必然 $\geq x$ ，因此必然会形成环，不可能构成树。

AGC029F Construction of a tree

首先考虑如果存在 x 个集合，他们的并的大小是 $\leq x$ ，那么必然无解。考虑这 x 个集合的并的点的子集在树上的连边数量必然 $\geq x$ ，因此必然会形成环，不可能构成树。

否则考虑在每个集合中选择一个点 x ，使得所有点至多出现在一个集合中，如果有解这样一定是能做到的：

AGC029F Construction of a tree

首先考虑如果存在 x 个集合，他们的并的大小是 $\leq x$ ，那么必然无解。考虑这 x 个集合的并的点的子集在树上的连边数量必然 $\geq x$ ，因此必然会形成环，不可能构成树。

否则考虑在每个集合中选择一个点 x ，使得所有点至多出现在一个集合中，如果有解这样一定是能做到的：

以剩下那个没有被选择的点为根，依次拓展，具体的，考虑每次找到一个集合，使得其代表元素仍未被选择，但集合中存在至少一个元素被选择，此时将这个集合的代表元素的父亲设为已经被选择的那个元素即可，这样必然构成一棵树。

AGC029F Construction of a tree

考虑什么时候会无法拓展，那此时剩下仍然未被选择的所有集合的元素并的大小必然 = 剩下元素个数，此时与第一个有解的条件矛盾，因此若满足第一个条件，则必然不可能存在无法拓展的情况。

AGC029F Construction of a tree

考虑什么时候会无法拓展，那此时剩下仍然未被选择的所有集合的元素并的大小必然 = 剩下元素个数，此时与第一个有解的条件矛盾，因此若满足第一个条件，则必然不可能存在无法拓展的情况。

复杂度瓶颈在于为每个集合找出互不相同的代表元素，可以通过二分图匹配求得，复杂度 $O(n\sqrt{m})$ ，其中 $m = \sum |s_i|$ 。

CF1770H Koxia, Mahiru and Winter Festival

给定一张 $n \times n$ 的网格图，再给定两个 $1 \sim n$ 的排列 p, q 。现在在所有 $(1, x)$ 的位置上都有一个人，要走到 (n, p_x) 位置，在所有 $(x, 1)$ 的位置上也有一个人，要走到 (q_x, n) 的位置。特殊的， $(1, 1)$ 其实有两个人。

你要给每个人安排一条路线，只能在网格图上的网格。构造一种方案，最小化所有边被经过的次数的最大值。

$n \leq 200$ 。

CF1770H Koxia, Mahiru and Winter Festival

如果 p 和 q 都是 $1, \dots, n$, 那么直着走答案就是 1。不难发现只有这种情况答案可能是 1, 否则猜测答案一定是 2。考虑构造。

CF1770H Koxia, Mahiru and Winter Festival

如果 p 和 q 都是 $1, \dots, n$, 那么直着走答案就是 1。不难发现只有这种情况答案可能是 1, 否则猜测答案一定是 2。考虑构造。

先考虑一种特殊情况: p 和 q 都是 $n, \dots, 1$ 如何构造, 通过手玩我们能研究出来一种方式。

CF1770H Koxia, Mahiru and Winter Festival

如果 p 和 q 都是 $1, \dots, n$, 那么直着走答案就是 1。不难发现只有这种情况答案可能是 1, 否则猜测答案一定是 2。考虑构造。

先考虑一种特殊情况: p 和 q 都是 $n, \dots, 1$ 如何构造, 通过手玩我们能研究出来一种方式。

然后考虑调整, 发现关键性质: 对于 $a = [n, \dots, 1]$, 我们可以通过若干次满足 $x < y, a_x > a_y$ 的交换 a_x, a_y 的操作, 将 a 变成任意的排列! 考虑如果我们要交换 $a_x > a_y$ 的两个值 a_x, a_y , 那么 $x \rightarrow a_x$ 和 $y \rightarrow a_y$ 的两条路径一定有交, 我们直接交换交点之后的路径即可。直接这样构造就能做到 $O(n^2)$ 的复杂度。

ARC189E Straight Path

给定 n ，你需要给 n 个点的无向完全图每条边染上一种颜色，使得不存在一条恰好经过所有点一次的路径，满足路径上的边权非严格递增。最小化染的颜色数量，或者报告无解。

$2 \leq n \leq 20$ 。

ARC189E Straight Path

首先，颜色数为 1 显然不行，颜色数为 2 也是不行的。考虑构造三种颜色的方案。

ARC189E Straight Path

首先，颜色数为 1 显然不行，颜色数为 2 也是不行的。考虑构造三种颜色的方案。

忽略特殊情况后考虑一种通用的构造方法：将所有点拆分成 4 个集合 S_0, S_1, S_2, S_3 ，满足 $|S_1| \geq |S_2| \geq |S_3| \geq |S_4| \geq |S_1| - 1$ ，也就是尽可能平均的将点分成四个集合。然后对于 $u \in S_x, v \in S_y$ ， (u, v) 边的颜色根据 (x, y) 讨论：

ARC189E Straight Path

首先，颜色数为 1 显然不行，颜色数为 2 也是不行的。考虑构造三种颜色的方案。

忽略特殊情况后考虑一种通用的构造方法：将所有点拆分成 4 个集合 S_0, S_1, S_2, S_3 ，满足 $|S_1| \geq |S_2| \geq |S_3| \geq |S_4| \geq |S_1| - 1$ ，也就是尽可能平均的将点分成四个集合。然后对于 $u \in S_x, v \in S_y$ ， (u, v) 边的颜色根据 (x, y) 讨论：

- ▶ (x, y) 为 $(0, 1)$ 或 $(2, 3)$ 时，染成 1 颜色；
- ▶ (x, y) 为 $(0, 3)$ 或 $(1, 2)$ 时，染成 2 颜色；
- ▶ 对于其他所有情况，染成 3 颜色。

ARC189E Straight Path

考虑为什么这样构造是对的。假设存在一条合法路径，起点在 S_0 中，那么这条路径只有以下两种情况：

ARC189E Straight Path

考虑为什么这样构造是对的。假设存在一条合法路径，起点在 S_0 中，那么这条路径只有以下两种情况：

- ▶ 先在 S_0 和 S_1 中穿梭，然后回到 S_0 ，然后在 S_0 和 S_3 之间穿梭，由于最后还要走 S_2 中的点，因此只能再次回到 S_0 ，然后在 S_0 和 S_2 之间穿梭；
- ▶ 先在 S_0 和 S_1 中穿梭，然后到 S_1 中结束，再在 S_1 和 S_2 中穿梭，由于还要走 S_3 中的点，因此只能走到 S_1 ，然后再在 S_1 和 S_3 之间穿梭。

ARC189E Straight Path

如果第一种路径存在，必须满足 $|S_0| \geq |S_1| + |S_3| + 1$ ，与前面的条件矛盾；如果第二种存在，则满足 $|S_1| \geq |S_0| + |S_2|$ ，也矛盾（注意此处分析是对于 n 较大的情况）。对于起点在其他集合中的情况也是类似的可以推出矛盾。

ARC189E Straight Path

如果第一种路径存在，必须满足 $|S_0| \geq |S_1| + |S_3| + 1$ ，与前面的条件矛盾；如果第二种存在，则满足 $|S_1| \geq |S_0| + |S_2|$ ，也矛盾（注意此处分析是对于 n 较大的情况）。对于起点在其他集合中的情况也是类似的可以推出矛盾。

但是 n 在较小时可能有 eps 的差距导致部分不等式可以满足，具体来说，通过搜索得到 $n = 2, 3$ 时无解， $n = 5$ 时需要 4 种颜色（样例已经给出了构造方法），剩余情况按照上述构造即可。

目录

构造选讲

非传统题选讲

交互选讲

通信选讲

提交答案选讲

随机与近似算法选讲

目录

构造选讲

非传统题选讲

交互选讲

通信选讲

提交答案选讲

随机与近似算法选讲

什么是交互？

交互题通常有一个交互库。你并不能获取题目输入的所有信息，通常你是需要猜测某一些信息。

什么是交互？

交互题通常有一个交互库。你并不能获取题目输入的所有信息，通常你是需要猜测某一些信息。

你可以调用的询问 / 操作，交互库会通过一个接口的形式提供给你（部分 IO 交互是通过标准输入输出，但本质也是给了一个接口）。

什么是交互？

交互题通常有一个交互库。你并不能获取题目输入的所有信息，通常你是需要猜测某一些信息。

你可以调用的询问 / 操作，交互库会通过一个接口的形式提供给你（部分 IO 交互是通过标准输入输出，但本质也是给了一个接口）。

你需要通过这些接口，来猜测出题目希望你猜测的信息。

什么是交互？

交互题通常有一个交互库。你并不能获取题目输入的所有信息，通常你是需要猜测某一些信息。

你可以调用的询问 / 操作，交互库会通过一个接口的形式提供给你（部分 IO 交互是通过标准输入输出，但本质也是给了一个接口）。

你需要通过这些接口，来猜测出题目希望你猜测的信息。

一些强制在线的题目也可能通过交互的形式进行，不过这一类题目本质并不是交互题。

你要猜一个长度为 n 的字符串 s 。可以进行下面这一种询问：

- ▶ 给定 l, r ，查询 $s[l \cdots r]$ 的所有子串打乱之后的结果（子串顺序打乱，每个子串内部也打乱）。

你可以最多进行 3 次询问，并且询问返回的子串总数不超过 $0.777(n+1)^2$ 。

$1 \leq n \leq 100$ ，部分分返回的子串总数不超过 $(n+1)^2$ 。

CF1286C2 Madhouse

部分分：直接询问 $[1, n - 1]$ 和 $[1, n]$ ，两次结果做查分可以得到 s 的所有后缀打乱的结果。按照长度即可还原。

CF1286C2 Madhouse

部分分：直接询问 $[1, n-1]$ 和 $[1, n]$ ，两次结果做查分可以得到 s 的所有后缀打乱的结果。按照长度即可还原。

先对 $[1, n/2]$ 做部分分做法，可以在 $0.25n^2$ 的子串总数内得到前半串。

部分分：直接询问 $[1, n-1]$ 和 $[1, n]$ ，两次结果做查分可以得到 s 的所有后缀打乱的结果。按照长度即可还原。

先对 $[1, n/2]$ 做部分分做法，可以在 $0.25n^2$ 的子串总数内得到前半串。

然后再询问 $[1, n]$ 。对于每个字符，观察他在长度为 i 的子串中出现的次数和在长度为 $i+1$ 的子串中的出现次数可以得到他在 $[i+1, n-i]$ 这一段的出现次数。

部分分：直接询问 $[1, n-1]$ 和 $[1, n]$ ，两次结果做查分可以得到 s 的所有后缀打乱的结果。按照长度即可还原。

先对 $[1, n/2]$ 做部分分做法，可以在 $0.25n^2$ 的子串总数内得到前半串。

然后再询问 $[1, n]$ 。对于每个字符，观察他在长度为 i 的子串中出现的次数和在长度为 $i+1$ 的子串中的出现次数可以得到他在 $[i+1, n-i]$ 这一段的出现次数。

由于已经知道了前半串，因此上面的信息足够还原整串。共三次询问，返回子串总数为 $0.75n^2$ 。

CF1370F2 The Hidden Pair

给定一棵 n 个节点的树，上面有两个隐藏节点 x, y 。

你可以进行下面的询问：

- ▶ 给定一个集合 S ，返回 $\min_{u \in S} d(u, x) + d(u, y)$ ，并且得到任意一个 $\arg \min_{u \in S} d(u, x) + d(u, y)$ 。

你需要在最多 11 次询问内猜出 x, y 。

$1 \leq n \leq 1000$ 。部分分：12 次询问。

CF1370F2 The Hidden Pair

先问一次 $S = \{1, 2, \dots, n\}$ 可以得到 $d(x, y)$ 以及任意一个 $x - y$ 路径上的点 r 。

CF1370F2 The Hidden Pair

先问一次 $S = \{1, 2, \dots, n\}$ 可以得到 $d(x, y)$ 以及任意一个 $x - y$ 路径上的点 r 。

以 r 为根，不妨设 $d(x, r) > d(y, r)$ 也就是 $dep_x > dep_y$ 。考虑二分得到 dep_x ：每次询问所有 $dep = mid$ 的节点，如果 $mid \leq x$ 则 $\min$ 恰好为 $d(x, y)$ 。并且二分结束时刚好能得到 x 。

CF1370F2 The Hidden Pair

先问一次 $S = \{1, 2, \dots, n\}$ 可以得到 $d(x, y)$ 以及任意一个 $x - y$ 路径上的点 r 。

以 r 为根，不妨设 $d(x, r) > d(y, r)$ 也就是 $dep_x > dep_y$ 。考虑二分得到 dep_x ：每次询问所有 $dep = mid$ 的节点，如果 $mid \leq x$ 则 $\min$ 恰好为 $d(x, y)$ 。并且二分结束时刚好能得到 x 。

然后在以 x 为根询问所有 $dep = d(x, y)$ 的节点就能找到 y 。

CF1370F2 The Hidden Pair

先问一次 $S = \{1, 2, \dots, n\}$ 可以得到 $d(x, y)$ 以及任意一个 $x - y$ 路径上的点 r 。

以 r 为根，不妨设 $d(x, r) > d(y, r)$ 也就是 $dep_x > dep_y$ 。考虑二分得到 dep_x ：每次询问所有 $dep = mid$ 的节点，如果 $mid \leq x$ 则 $\min$ 恰好为 $d(x, y)$ 。并且二分结束时刚好能得到 x 。

然后在以 x 为根询问所有 $dep = d(x, y)$ 的节点就能找到 y 。

询问次数 $1 + \log n + 1 = 12$ ，可以通过部分分。

CF1370F2 The Hidden Pair

优化方法：由于 dep_x 一定在 $\frac{1}{2}d(x, y)$ 和 $d(x, y)$ 之间，因此二分的范围可以至少缩减到 $\frac{1}{2}n$ ，少一次询问。

CF1442F Differentiating Games

给定一张 n 个点 m 条边的有向无环图。你可以先进行至多 4242 次加删边操作，操作后的图不一定需要是 DAG。

然后有一个隐藏节点 x 。每次询问你可以再给定一个集合 S 。交互库会告诉你如果初始时在 S 中的点和 x 各放一个棋子，做有向图博弈，最后的结果是先手必胜/先手必败/平局。

你需要在至多 20 次询问，询问 S 大小之和 ≤ 20 限制下求出 x 。

$1 \leq n \leq 1000$, $0 \leq m \leq 10^5$ 。

CF1442F Differentiating Games

最后变成的图肯定不能是 DAG，不然让所有点的 SG 值互不相同都没法做到。

CF1442F Differentiating Games

最后变成的图肯定不能是 DAG，不然让所有点的 SG 值互不相同都没法做到。

因此考虑造自环。具体构造方式如下：选 20 个点构成集合 S ，把 S 造成一个 20 个点的完全 DAG，并写 S 不向外面任何节点连边。

CF1442F Differentiating Games

最后变成的图肯定不能是 DAG，不然让所有点的 SG 值互不相同都没法做到。

因此考虑造自环。具体构造方式如下：选 20 个点构成集合 S ，把 S 造成一个 20 个点的完全 DAG，并写 S 不向外面任何节点连边。

然后对于 $U \setminus S$ 中的每个节点 u ，先连一个子环，然后选一个 S 的子集 T_u ，向 T_u 中的点连边，不向 $S \setminus T_u$ 中的点连边。

CF1442F Differentiating Games

每次询问时，只询问一个节点 u 。考虑三种结果：

CF1442F Differentiating Games

每次询问时，只询问一个节点 u 。考虑三种结果：

- ▶ 如果先手必败，则 $u = x$ 。
- ▶ 如果先手必胜，则 $x \in S$ 或 $u \in T_x$ 。
- ▶ 如果平局，则 $u \notin T_x$ 。

CF1442F Differentiating Games

每次询问时，只询问一个节点 u 。考虑三种结果：

- ▶ 如果先手必败，则 $u = x$ 。
- ▶ 如果先手必胜，则 $x \in S$ 或 $u \in T_x$ 。
- ▶ 如果平局，则 $u \notin T_x$ 。

这样只要 T_u 互不相同，只需要依次询问 S 中的 20 个节点就能得到 x 。

CF1442F Differentiating Games

最后考虑如何构造这样的图。

CF1442F Differentiating Games

最后考虑如何构造这样的图。

随便求一个拓扑序，然后选取最后的 20 个节点作为 S ，自然 S 不会向外面连边。

CF1442F Differentiating Games

最后考虑如何构造这样的图。

随便求一个拓扑序，然后选取最后的 20 个节点作为 S ，自然 S 不会向外面连边。

这样构造完全 DAG 最多花费 $\binom{20}{2} = 190$ 次操作。

CF1442F Differentiating Games

最后考虑如何构造这样的图。

随便求一个拓扑序，然后选取最后的 20 个节点作为 S ，自然 S 不会向外面连边。

这样构造完全 DAG 最多花费 $\binom{20}{2} = 190$ 次操作。

剩下 980 个节点连子环需要 980 次操作。由于 $\binom{20}{3} = 1140 > 1000$ ，因此每个点至多只需要 flip 3 条边的状态就可以构造出互不相同的 T_u 。

CF1442F Differentiating Games

最后考虑如何构造这样的图。

随便求一个拓扑序，然后选取最后的 20 个节点作为 S ，自然 S 不会向外面连边。

这样构造完全 DAG 最多花费 $\binom{20}{2} = 190$ 次操作。

剩下 980 个节点连子环需要 980 次操作。由于 $\binom{20}{3} = 1140 > 1000$ ，因此每个点至多只需要 flip 3 条边的状态就可以构造出互不相同的 T_u 。

总次数 $190 + 980 \times (1 + 3) = 4110$ 次符合限制。

目录

构造选讲

非传统题选讲

交互选讲

通信选讲

提交答案选讲

随机与近似算法选讲

什么是通信？

一道普通的算法竞赛题目是通过题目给定一些输入信息，你需要产出一些输出信息。交互题也是，只不过这个过程可能会进行多轮。本质上，这一类题目都是你在与 `judger` 对话。

什么是通信？

一道普通的算法竞赛题目是通过题目给定一些输入信息，你需要产出一些输出信息。交互题也是，只不过这个过程可能会进行多轮。本质上，这一类题目都是你在与 `judger` 对话。

但是在通信题中，你不仅需要与 `judger` 进行对话，通常你还需要实现多个程序，在程序之间也进行信息的交互。

什么是通信？

一道普通的算法竞赛题目是通过题目给定一些输入信息，你需要产出一些输出信息。交互题也是，只不过这个过程可能会进行多轮。本质上，这一类题目都是你在与 `judger` 对话。

但是在通信题中，你不仅需要与 `judger` 进行对话，通常你还需要实现多个程序，在程序之间也进行信息的交互。

对于一个常规的通信题，你需要同时实现一个 `encoder` 和一个 `decoder`。`encoder` 读入题目信息，你需要设计一种方式对信息进行转化；`decoder` 则是得到你转化后的信息，并尝试得到题目需要的输出信息。

什么是通信？

一道普通的算法竞赛题目是通过题目给定一些输入信息，你需要产出一些输出信息。交互题也是，只不过这个过程可能会进行多轮。本质上，这一类题目都是你在与 `judger` 对话。

但是在通信题中，你不仅需要与 `judger` 进行对话，通常你还需要实现多个程序，在程序之间也进行信息的交互。

对于一个常规的通信题，你需要同时实现一个 `encoder` 和一个 `decoder`。`encoder` 读入题目信息，你需要设计一种方式对信息进行转化；`decoder` 则是得到你转化后的信息，并尝试得到题目需要的输出信息。

通常来说，通信题的评分关注的是你转化后信息的复杂度。

QOJ141 8 染色

给定一张 n 个点 m 条边的无向图，保证其可以八染色（给每个点染 $0 \sim 7$ 之间的颜色，且每条边连接的两个节点颜色不同）。

encoder：得到一种染色方案，可以返回一个长度不超过 2.5×10^5 的 01 串。

decoder：得到 encoder 的 01 串，返回任意一种合法染色方案。

$1 \leq n \leq 2 \times 10^5$, $0 \leq m \leq 5 \times 10^5$ 。

QOJ141 8 染色

由于二染色是简单的，一个简单的想法是把每个点都颜色传 2 个 bit 过去，最后一个 bit 用 2 染色随便还原一种。这样串长为 $2n$ 。

QOJ141 8 染色

由于二染色是简单的，一个简单的想法是把每个点都颜色传 2 个 bit 过去，最后一个 bit 用 2 染色随便还原一种。这样串长为 $2n$ 。

注意到如果每个点的度数都 ≤ 7 ，那么按照任意一种顺序染色，每次染所有邻居颜色都 mex 即可。

QOJ141 8 染色

由于二染色是简单的，一个简单的想法是把每个点都颜色传 2 个 bit 过去，最后一个 bit 用 2 染色随便还原一种。这样串长为 $2n$ 。

注意到如果每个点的度数都 ≤ 7 ，那么按照任意一种顺序染色，每次染所有邻居颜色都 mex 即可。

因此其实我们只需要关心所有 $\deg \geq 8$ 的节点即可。这样的节点至多只有 $\frac{2m}{8} = \frac{1}{4}m$ 个，每次点传递 2 bit。总串长 $\frac{1}{2}m$ 符合限制。

P11984 [JOIST 2025] 占卜 3

Anna 和 Bruno 在玩一种用卡牌进行的占卜游戏 Q 次，规则如下。

- ▶ 有许多上面写着 0 或 1 的卡牌叠成一堆。Anna 从牌堆中一次抽出 $N (= 900)$ 张卡牌。Anna 和 Bruno 知道 N 的值。
- ▶ 每次抽到卡牌时，她会决定是弃掉这张牌还是将其放到桌上。
- ▶ ▶ 如果她选择将牌放到桌上，她会把这张牌插入桌上的卡牌序列中。
 - ▶ 形式化地，当桌上有 l 张卡牌时，她选定非负整数 x ($0 \leq x \leq l$)，并将这张牌放在桌上从左数第 x 张卡牌右侧。
 - ▶ 当 $x = 0$ 时，她将这张牌放在桌上卡牌序列的最左侧。

P11984 [JOIST 2025] 占卜 3

- ▶ 当 Anna 抽完并处理完 N 张卡牌后，她的操作结束。占卜结果是这 N 张卡牌中上面写着数字 1 的卡牌数量。
- ▶ 当 Anna 结束操作后，Bruno 会看到桌上的卡牌序列。基于这些信息，他需要猜出占卜结果。

如果 Bruno 猜对了，占卜就算成功。当桌上的卡牌越少时，占卜被认为越高明。

满分： $L \leq 14$ 。有意思的分： $L \leq 18$ 。

P11984 [JOIST 2025] 占卜 3

一个没什么用的做法：每拿到两张牌考虑一下，如果颜色相同就把第二张留下，否则全部不留下。这样可以算出 1 的个数。最坏情况下留 $\frac{1}{2}n$ 张。

P11984 [JOIST 2025] 占卜 3

一个没什么用的做法：每拿到两张牌考虑一下，如果颜色相同就把第二张留下，否则全部不留下。这样可以算出 1 的个数。最坏情况下留 $\frac{1}{2}n$ 张。

考虑用最后 10 张牌来表示信息，前面的牌只负责构造一个模式。

P11984 [JOIST 2025] 占卜 3

一个没什么用的做法：每拿到两张牌考虑一下，如果颜色相同就把第二张留下，否则全部不留下。这样可以算出 1 的个数。最坏情况下留 $\frac{1}{2}n$ 张。

考虑用最后 10 张牌来表示信息，前面的牌只负责构造一个模式。

具体来说，考虑一种特殊情况：最后 10 张牌全是 1。此时我们对于前面的 890 张牌，只选择任意的 4 张 0 留下来，同时记录一下当前 1 的总个数。然后用最后 10 个 1 和前面的 4 个 0 共可以表示出 $\binom{14}{4} = 1001$ 种不同状态，足够表达一个 ≤ 890 的整数了。

P11984 [JOIST 2025] 占卜 3

问题是最后 10 张牌并不一定全部是 1。因此在保留前缀模式的时候考虑多保留一些。具体来说，保留 8 张，其中 4 张 0 和 4 张 1，排成 00001111 的模式。

P11984 [JOIST 2025] 占卜 3

问题是最后 10 张牌并不一定全部是 1。因此在保留前缀模式的时候考虑多保留一些。具体来说，保留 8 张，其中 4 张 0 和 4 张 1，排成 00001111 的模式。

对于最后 10 张牌，依旧全部保留。对于一张 0，把它插到后面的 1 那边；对于一张 1，把它插到前面的 0 那边。这样 Bob 在解析的时候找前 4 个 0 的位置就可以定位到哪些 1 是最后 10 张里面插进去的。

P11984 [JOIST 2025] 占卜 3

问题是最后 10 张牌并不一定全部是 1。因此在保留前缀模式的时候考虑多保留一些。具体来说，保留 8 张，其中 4 张 0 和 4 张 1，排成 00001111 的模式。

对于最后 10 张牌，依旧全部保留。对于一张 0，把它插到后面的 1 那边；对于一张 1，把它插到前面的 0 那边。这样 Bob 在解析的时候找前 4 个 0 的位置就可以定位到哪些 1 是最后 10 张里面插进去的。

这样表示的信息也是足够的。特殊情况是前 890 张牌里面不足 4 个 0 或 4 个 1，但这是容易特判的。

P11984 [JOIST 2025] 占卜 3

问题是最后 10 张牌并不一定全部是 1。因此在保留前缀模式的时候考虑多保留一些。具体来说，保留 8 张，其中 4 张 0 和 4 张 1，排成 00001111 的模式。

对于最后 10 张牌，依旧全部保留。对于一张 0，把它插到后面的 1 那边；对于一张 1，把它插到前面的 0 那边。这样 Bob 在解析的时候找前 4 个 0 的位置就可以定位到哪些 1 是最后 10 张里面插进去的。

这样表示的信息也是足够的。特殊情况是前 890 张牌里面不足 4 个 0 或 4 个 1，但这是容易特判的。

这样最多保留 18 张牌。优化到 14 的过程很无趣，这里不讲。

目录

构造选讲

非传统题选讲

交互选讲

通信选讲

提交答案选讲

随机与近似算法选讲

什么是提交答案？

通常来说，对于一道算法竞赛题目，你需要提交的是一份可以在特定环境下被编译的源代码。

什么是提交答案？

通常来说，对于一道算法竞赛题目，你需要提交的是一份可以在特定环境下被编译的源代码。

但是在提交答案题中，你需要提交的是若干输出文件。

什么是提交答案？

通常来说，对于一道算法竞赛题目，你需要提交的是一份可以在特定环境下被编译的源代码。

但是在提交答案题中，你需要提交的是若干输出文件。

为什么要这么设计？原因大概有几个：

什么是提交答案？

通常来说，对于一道算法竞赛题目，你需要提交的是一份可以在特定环境下被编译的源代码。

但是在提交答案题中，你需要提交的是若干输出文件。

为什么要这么设计？原因大概有几个：

- ▶ 题目设计的标准算法可能是启发式算法，需要较长的运行时间来让算法收敛到一个不错的解；
- ▶ 题目设计的标准算法可能是搜索，考察你的剪枝程度，需要较长运行时间；
- ▶ 题目的输入具有一定的特殊性质，需要你自行通过输入观察性质；
- ▶ 出题人想整活。。

P4920 [WC2015] 未来程序

给定 10 个程序和 10 个输入，你要求出 10 个输出。

QOJ10877 人工智障

给定一份编译好的可执行文件，有 10 个不同模式。

对于每一个模式，给定了一个输出文件，你不知道这个模式求的是什么，你需要构造一个输入使程序能够输出给定的文件。

P13536 [IOI 2025] 神话三峰

对于一个长度为 n 的正整数序列 h ，定义一个三元组 (i, j, k) 是好的，当且仅当 $i < j < k$ ，且 $\{j - i, k - j, k - i\} = \{h_i, h_j, h_k\}$ 。

第一问：给定一个序列，求有多少个好的三元组。 $3 \leq n \leq 2 \cdot 10^5$ 。

第二问：构造一个长度为 $2 \cdot 10^5$ ，有至少 $1.2 \cdot 10^7$ 个好的三元组的序列（提交答案）。

P13536 [IOI 2025] 神话三峰

先看第一问。考虑 $\{j-i, k-j, k-i\}$ 和 $\{h_i, h_j, h_k\}$ 的具体对应关系。如果 h_i 或者 h_k 对应 $k-i$ ，那么是容易统计的。

P13536 [IOI 2025] 神话三峰

先看第一问。考虑 $\{j-i, k-j, k-i\}$ 和 $\{h_i, h_j, h_k\}$ 的具体对应关系。如果 h_i 或者 h_k 对应 $k-i$ ，那么是容易统计的。

否则 h_j 对应 $k-i$ 。如果 h_i 对应 $j-i$ 且 h_k 对应 $k-j$ ，那么也是简单的。

P13536 [IOI 2025] 神话三峰

先看第一问。考虑 $\{j-i, k-j, k-i\}$ 和 $\{h_i, h_j, h_k\}$ 的具体对应关系。如果 h_i 或者 h_k 对应 $k-i$ ，那么是容易统计的。

否则 h_j 对应 $k-i$ 。如果 h_i 对应 $j-i$ 且 h_k 对应 $k-j$ ，那么也是简单的。

唯一复杂的情况是 h_j 对应 $k-i$ ， h_i 对应 $k-j$ ， h_k 对应 $j-i$ 。对于这种情况，把条件改写成：

$$\begin{cases} j - h_k = k - h_j \\ j + h_i = i + h_j \\ i + h_k = k - h_i \end{cases}$$

P13536 [IOI 2025] 神话三峰

等价于

$$\begin{cases} j + h_j = k + h_k \\ j - h_j = i - h_i \\ i + h_i = k - h_k \end{cases}$$

P13536 [IOI 2025] 神话三峰

等价于

$$\begin{cases} j + h_j = k + h_k \\ j - h_j = i - h_i \\ i + h_i = k - h_k \end{cases}$$

考虑在 $i + h_i, i - h_i$ 两个点之间连边，那么上面的条件等价于 i, j, k 三条边组成的三元环。并且一个三元环恰好对应一个合法的三元组。

P13536 [IOI 2025] 神话三峰

等价于

$$\begin{cases} j + h_j = k + h_k \\ j - h_j = i - h_i \\ i + h_i = k - h_k \end{cases}$$

考虑在 $i + h_i, i - h_i$ 两个点之间连边，那么上面的条件等价于 i, j, k 三条边组成的三元环。并且一个三元环恰好对应一个合法的三元组。

因此问题被转化为了 $2 \cdot 10^5$ 条边的图的三元环计数。容易做到 $O(n\sqrt{n})$ 。第一问解决。

P13536 [IOI 2025] 神话三峰

考虑第二问，前面几种情况所产生的合法三元组数量是 $O(n)$ 的，肯定无法达到要求。

P13536 [IOI 2025] 神话三峰

考虑第二问，前面几种情况所产生的合法三元组数量是 $O(n)$ 的，肯定无法达到要求。

因此只能对着最后一种情况构造。因此问题本质上是要找一个 n 条边的图，满足每条边的两个端点编号之和恰好对应 $[0, 2(n-1)]$ 之间的两两不同的偶数，并且让三元环尽可能多。

P13536 [IOI 2025] 神话三峰

考虑第二问，前面几种情况所产生的合法三元组数量是 $O(n)$ 的，肯定无法达到要求。

因此只能对着最后一种情况构造。因此问题本质上是要找一个 n 条边的图，满足每条边的两个端点编号之和恰好对应 $[0, 2(n-1)]$ 之间的两两不同的偶数，并且让三元环尽可能多。

一种构造方式是：初始只有一个点 0，每次选择一个能让增加边的数量最多的点编号加入进去，并连上所有能加的边，一直重复直到无法加入。

P13536 [IOI 2025] 神话三峰

考虑第二问，前面几种情况所产生的合法三元组数量是 $O(n)$ 的，肯定无法达到要求。

因此只能对着最后一种情况构造。因此问题本质上是要找一个 n 条边的图，满足每条边的两个端点编号之和恰好对应 $[0, 2(n-1)]$ 之间的两两不同的偶数，并且让三元环尽可能多。

一种构造方式是：初始只有一个点 0，每次选择一个能让增加边的数量最多的点编号加入进去，并连上所有能加的边，一直重复直到无法加入。

跑一下发现在 $n = 2 \cdot 10^5$ 时能够产生超过 1.5×10^7 个三元环。

目录

构造选讲

非传统题选讲

交互选讲

通信选讲

提交答案选讲

随机与近似算法选讲

什么是随机算法？

算法具有随机性。即有一定概率可以获得正确的解。

什么是随机算法？

算法具有随机性。即有一定概率可以获得正确的解。

只要你的算法具有足够高的正确率（例如哈希只有 $1/P$ 的错误率），那么就可以近似认为该算法在算法竞赛中是可以被接受的。

什么是随机算法？

算法具有随机性。即有一定概率可以获得正确的解。

只要你的算法具有足够高的正确率（例如哈希只有 $1/P$ 的错误率），那么就可以近似认为该算法在算法竞赛中是可以被接受的。

对于一个二分类问题，如果一个随机算法只有可能把阳性判断成阴性（或者把阴性判断成阳性），那么该算法只需要有常数的正确概率，就可以通过多次运行来提高到足够高的正确率。

什么是近似算法？

对于一个问题，我们可能无法求出其精确解。

什么是近似算法?

对于一个问题，我们可能无法求出其精确解。

但是，我们可以在比较优秀的复杂度内求出一组在精确解附近的解。最简单的例子：我们很容易能求出一个点集直径的 $2 -$ 近似解。

什么是近似算法？

对于一个问题，我们可能无法求出其精确解。

但是，我们可以在比较优秀的复杂度内求出一组在精确解附近的解。最简单的例子：我们很容易能求出一个点集直径的 2 -近似解。

很多时候，近似算法中也需要用到随机化的思想。

矩阵乘法测试

给定三个 $n \times n$ 的矩阵 A, B, C ，你需要判断 $A \times B = C$ 是否成立。要求复杂度 $O(n^2)$ 。

矩阵乘法测试

给定三个 $n \times n$ 的矩阵 A, B, C ，你需要判断 $A \times B = C$ 是否成立。要求复杂度 $O(n^2)$ 。

做法：随机一个 n 维向量 x ，判断 $ABx = Cx$ 是否成立即可。
 $ABx = A(Bx)$ 可以 $O(n^2)$ 计算。

Rejection Sampling

你有一枚质量均匀的硬币，你需要使用它等概率生成一个 $\{0, 1, 2\}$ 中的随机数。

Rejection Sampling

你有一枚质量均匀的硬币，你需要使用它等概率生成一个 $\{0, 1, 2\}$ 中的随机数。

做法：抛两次，一共有 HH, HT, TH, TT 四种可能情况。将前三种情况分别对应选择 0, 1, 2，第四种情况则重新抛。这样期望抛 $O(1)$ 轮能得到结果。

Rejection Sampling

你有一枚质量均匀的硬币，你需要使用它等概率生成一个 $\{0, 1, 2\}$ 中的随机数。

做法：抛两次，一共有 HH, HT, TH, TT 四种可能情况。将前三种情况分别对应选择 0, 1, 2，第四种情况则重新抛。这样期望抛 $O(1)$ 轮能得到结果。

扩展：对于任意实数 $p \in [0, 1]$ ，以 p 的概率生成 0， $1 - p$ 的概率生成 1。

Rejection Sampling

你有一枚质量均匀的硬币，你需要使用它等概率生成一个 $\{0, 1, 2\}$ 中的随机数。

做法：抛两次，一共有 HH, HT, TH, TT 四种可能情况。将前三种情况分别对应选择 0, 1, 2，第四种情况则重新抛。这样期望抛 $O(1)$ 轮能得到结果。

扩展：对于任意实数 $p \in [0, 1]$ ，以 p 的概率生成 0， $1 - p$ 的概率生成 1。

做法：考虑 p 的二进制小数表示，逐位做，每一位有 $1/2$ 的概率成功得到结果。

Count-min Sketch

设计一个数据结构，支持对于任意的误差参数 ϵ ，都能做：

- ▶ 插入 N 个 $1 \sim n$ 之间的整数；
- ▶ 给定一个整数 x ，查询 x 的出现次数。

要求查询得到的结果大概率满足 $C_x \leq \hat{C} \leq C_x + \epsilon \cdot N$ 。

要求空间复杂度 $\frac{\text{poly log } n}{\epsilon}$ 。

Count-min Sketch

设计一个数据结构，支持对于任意的误差参数 ϵ ，都能做：

- ▶ 插入 N 个 $1 \sim n$ 之间的整数；
- ▶ 给定一个整数 x ，查询 x 的出现次数。

要求查询得到的结果大概率满足 $C_x \leq \hat{C} \leq C_x + \epsilon \cdot N$ 。

要求空间复杂度 $\frac{\text{poly log } n}{\epsilon}$ 。

做法：设计 k 个独立的哈希函数 $h_i: [n] \rightarrow [m]$ ，然后开 k 个大小为 m 的桶，每次插入在哈希到的位置，查询结果为所有桶对应位置的出现次数 $\min$ 。

Count-min Sketch

设计一个数据结构，支持对于任意的误差参数 ϵ ，都能做：

- ▶ 插入 N 个 $1 \sim n$ 之间的整数；
- ▶ 给定一个整数 x ，查询 x 的出现次数。

要求查询得到的结果大概率满足 $C_x \leq \hat{C} \leq C_x + \epsilon \cdot N$ 。

要求空间复杂度 $\frac{\text{poly log } n}{\epsilon}$ 。

做法：设计 k 个独立的哈希函数 $h_i: [n] \rightarrow [m]$ ，然后开 k 个大小为 m 的桶，每次插入在哈希到的位置，查询结果为所有桶对应位置的出现次数 $\min$ 。

取 $m = \frac{2}{\epsilon}$ ， $k = O(\log n)$ 即可有较高正确率。

Minhash

定义两个集合 A, B 的相似度 $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$ 。

给定 n 个大小为 m 的集合 S_i 。 q 次询问，每次给定 x, y ，近似估计 $J(S_x, S_y)$ 。要求复杂度 $O(nm + q)$ 。

Minhash

定义两个集合 A, B 的相似度 $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$ 。

给定 n 个大小为 m 的集合 S_i 。 q 次询问，每次给定 x, y ，近似估计 $J(S_x, S_y)$ 。要求复杂度 $O(nm + q)$ 。

解法：对每一个元素 x 随机赋权 v_x 。定义一个集合的哈希值 $h(A) = \min_{x \in A} v_x$ 。

Minhash

定义两个集合 A, B 的相似度 $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$ 。

给定 n 个大小为 m 的集合 S_i 。 q 次询问，每次给定 x, y ，近似估计 $J(S_x, S_y)$ 。要求复杂度 $O(nm + q)$ 。

解法：对每一个元素 x 随机赋权 v_x 。定义一个集合的哈希值 $h(A) = \min_{x \in A} v_x$ 。

则 $J(A, B) = \Pr[h(A) = h(B)]$ 。多次随机计算赋权 $h(A) = h(B)$ 的概率即可。

P4581 [BJOI2014] 想法

一共有 n 个集合，每个集合包含 $1 \sim m$ 之间的一些元素。第 $1 \leq i \leq m$ 个集合仅包含一个元素 i 。对于其他每个集合，是由前面给定的两个集合取并集得到的。

现在你需要估计每个集合的大小。要求 95% 以上的集合正确率达到 25%（即估计大小在 $[0.8X, 1.25X]$ 之间）。

$1 \leq m \leq 10^5$, $1 \leq n \leq 10^6$ 。

P4581 [BJOI2014] 想法

类似于上面的 Minhash。

P4581 [BJOI2014] 想法

类似于上面的 Minhash。

对于每个数随机一个 $[0, 1]$ 之间的实数权值。则 k 个这样随机的实数，其中最小值的期望是 $\frac{1}{k+1}$ 。

P4581 [BJOI2014] 想法

类似于上面的 Minhash。

对于每个数随机一个 $[0, 1]$ 之间的实数权值。则 k 个这样随机的实数，其中最小值的期望是 $\frac{1}{k+1}$ 。

操作是集合取并集，因此维护最小值的更新是简单的。

P4581 [BJOI2014] 想法

类似于上面的 Minhash。

对于每个数随机一个 $[0, 1]$ 之间的实数权值。则 k 个这样随机的实数，其中最小值的期望是 $\frac{1}{k+1}$ 。

操作是集合取并集，因此维护最小值的更新是简单的。

根据大数定律，可以用频率代替概率计算出最小值的大致期望，然后再用最小值的期望还原出集合大小即可。

P4581 [BJOI2014] 想法

类似于上面的 Minhash。

对于每个数随机一个 $[0, 1]$ 之间的实数权值。则 k 个这样随机的实数，其中最小值的期望是 $\frac{1}{k+1}$ 。

操作是集合取并集，因此维护最小值的更新是简单的。

根据大数定律，可以用频率代替概率计算出最小值的大致期望，然后再用最小值的期望还原出集合大小即可。

时间复杂度 $O(nT)$ ， T 是随机次数。

最小半包围圆 2-近似

给定二维平面上的 n 个点，求一个半径最小的圆，使得该圆内至少包含了 $\frac{n}{2}$ 个点。

你只需要求出答案的 2-近似即可。具体的：设符合条件的半径最小的圆半径为 R ，你只需要找出任意一个半径 $\leq 2R$ 的合法圆即可。

$$1 \leq n \leq 10^5。$$

最小半包围圆 2-近似

设最优答案的圆为 O ，半径为 R ，则直径为 $2R$ 。

最小半包围圆 2-近似

设最优答案的圆为 O ，半径为 R ，则直径为 $2R$ 。

也就是说，对于 O 的任意两个点，他们之间的距离 $\leq 2R$ 。

最小半包围圆 2-近似

设最优答案的圆为 O ，半径为 R ，则直径为 $2R$ 。

也就是说，对于 O 的任意两个点，他们之间的距离 $\leq 2R$ 。

那么我们以其中的任意一个点为圆心，取离这个点最近的 $\frac{n}{2}$ 个点作为圆内的点，就可以求出一个最小半包围圆的 2-近似。

最小半包围圆 2-近似

设最优答案的圆为 O ，半径为 R ，则直径为 $2R$ 。

也就是说，对于 O 的任意两个点，他们之间的距离 $\leq 2R$ 。

那么我们以其中的任意一个点为圆心，取离这个点最近的 $\frac{n}{2}$ 个点作为圆内的点，就可以求出一个最小半包围圆的 2-近似。

对于圆心的选择：由于最优答案包含了至少一半的点，因此随机选择一个点有 $\frac{1}{2}$ 的概率在圆内。随机常数次即可。

最小半包围圆 2-近似

设最优答案的圆为 O ，半径为 R ，则直径为 $2R$ 。

也就是说，对于 O 的任意两个点，他们之间的距离 $\leq 2R$ 。

那么我们以其中的任意一个点为圆心，取离这个点最近的 $\frac{n}{2}$ 个点作为圆内的点，就可以求出一个最小半包围圆的 2-近似。

对于圆心的选择：由于最优答案包含了至少一半的点，因此随机选择一个点有 $\frac{1}{2}$ 的概率在圆内。随机常数次即可。

复杂度 $O(Tn)$ ，其中 T 为随机次数。

最小半包围圆 $(1+\epsilon)$ -近似

给定二维平面上的 n 个点，求一个半径最小的圆，使得该圆内至少包含了 $\frac{n}{2}$ 个点。

你只需要求出答案的 $(1+\epsilon)$ -近似即可。具体的：设符合条件的半径最小的圆半径为 R ，你只需要找出任意一个半径 $\leq (1 + \epsilon)R$ 的合法圆即可。

$1 \leq n \leq 10^5$, $0.1 \leq \epsilon \leq 1$ 。

最小半包围圆 $(1+\epsilon)$ -近似

考虑先运行 2-近似的算法，得到一个 R ，满足最后答案的半径一定在 $[\frac{1}{2}R, R]$ 之间。

最小半包围圆 $(1+\epsilon)$ -近似

考虑先运行 2-近似的算法，得到一个 R ，满足最后答案的半径一定在 $[\frac{1}{2}R, R]$ 之间。

考虑将平面按照 $l \times l$ 的大小画格子，将所有点离散化到格点上，并且我们要求最后答案的圆心也在格点上。考虑以下两个事实：

最小半包围圆 $(1+\epsilon)$ -近似

考虑先运行 2-近似的算法，得到一个 R ，满足最后答案的半径一定在 $[\frac{1}{2}R, R]$ 之间。

考虑将平面按照 $l \times l$ 的大小画格子，将所有点离散化到格点上，并且我们要求最后答案的圆心也在格点上。考虑以下两个事实：

- ▶ 若 r 为原问题的合法答案半径，则离散化后的问题一定存在一个半径为 $r + \sqrt{2}l$ 的答案；
- ▶ 离散化后的解在半径增加 $\sqrt{2}l$ 之后一定能够覆盖原来的数据集。

最小半包围圆 $(1+\epsilon)$ -近似

考虑先运行 2-近似的算法，得到一个 R ，满足最后答案的半径一定在 $[\frac{1}{2}R, R]$ 之间。

考虑将平面按照 $l \times l$ 的大小画格子，将所有点离散化到格点上，并且我们要求最后答案的圆心也在格点上。考虑以下两个事实：

- ▶ 若 r 为原问题的合法答案半径，则离散化后的问题一定存在一个半径为 $r + \sqrt{2}l$ 的答案；
- ▶ 离散化后的解在半径增加 $\sqrt{2}l$ 之后一定能够覆盖原来的数据集。

取 $l = \frac{\sqrt{2}}{16}\epsilon R$ ，即可把原问题转化到离散化的格点上。

最小半包围圆 $(1+\epsilon)$ -近似

关键性质：在离散化后的网格图上，一个半径为 R 的圆至多只会能覆盖 $O(\frac{1}{\epsilon^2})$ 个格点。

最小半包围圆 $(1+\epsilon)$ -近似

关键性质：在离散化后的网格图上，一个半径为 R 的圆至多只会能覆盖 $O(\frac{1}{\epsilon^2})$ 个格点。

依旧随机一个点，假设这个点在最终答案的圆内。然后去枚举以这个点为圆心，半径为 R 的圆内的所有格点。

最小半包围圆 $(1+\epsilon)$ -近似

关键性质：在离散化后的网格图上，一个半径为 R 的圆至多只会能覆盖 $O(\frac{1}{\epsilon^2})$ 个格点。

依旧随机一个点，假设这个点在最终答案的圆内。然后去枚举以这个点为圆心，半径为 R 的圆内的所有格点。

以这些格点依次作为圆心，再去枚举其半径为 R 的圆内的所有格点，算出对应的最小的满足半包围条件的半径。

最小半包围圆 $(1+\epsilon)$ -近似

关键性质：在离散化后的网格图上，一个半径为 R 的圆至多只会能覆盖 $O(\frac{1}{\epsilon^2})$ 个格点。

依旧随机一个点，假设这个点在最终答案的圆内。然后去枚举以这个点为圆心，半径为 R 的圆内的所有格点。

以这些格点依次作为圆心，再去枚举其半径为 R 的圆内的所有格点，算出对应的最小的满足半包围条件的半径。

时间复杂度 $O(T(n + \frac{1}{\epsilon^2}))$ ，其中 T 为随机次数。

谢谢。